

44 - Паспорт приложения и экспериментальная эксплуатация (v0.9.1)

#request-processor #паспорт #эксплуатация #бой #оператор #разработка #2026-07-15

Паспорт приложения request-processor (экспериментальная эксплуатация)

Версия: 0.9.1

Дата паспорта: 2026-07-15

Репозиторий: https://github.com/shocknik/request_processor

Локально (dev): D:\My_projects\request_processor

Связано: 43 - Боевой запуск на рабочий ПК (v0.9.1) · 41 - Боевой опыт перенос между ПК (13.07.2026) · 40 - LLM Ollama статус план упаковки и согласование (13.07.2026) · 37 - Концепция продукта и роль ИИ (09.07.2026) · INSTALL.md (в репозитории)

Это документ **шире** инструкции оператора: зачем система устроена так, как устроена, что считать успехом экспериментальной эксплуатации, как не потерять опыт и не сломать прайс.

1. Идентификация

Поле	Значение
Название	request-processor / «Обработка заявок на испытания кабелей»
Назначение	Автоматизация цикла: входящая заявка → извлечение данных → проверка человеком → расчёт стоимости → КП / заявка на испытания / пакет документов
Платформа	Windows, Python ≥ 3.10, GUI (tkinter) + CLI (Click)
Хранение	SQLite (data/app.db), файлы в data/
Статус	Экспериментальная эксплуатация (не «закрытый продукт навсегда»)
Лицензия	MIT

Режим экспериментальной эксплуатации

Цель: ежедневно обрабатывать **реальные** заявки, накапливать **корректные** марки и организации, собирать **боевой опыт** для доработки парсера — без IDE на рабочем ПК.

Не цель v0.9.1: полный автопилот, Telegram, облачный LLM, fine-tune, 100% OCR на любых сканах.

2. Системные требования

Минимальные (ядро без LLM)

Компонент	Требование
ОС	Windows 10/11
CPU / RAM	обычный офисный ПК (4+ ГБ RAM)
Диск	~500 МБ на приложение + venv; место под generated/ , ocr_cache/
Python	3.10+ (рекомендуется 3.11/3.12), галочка Add to PATH
Tesseract	rus + eng — для PDF-сканов (для Word не обязателен)
Прайс	таблица test_items в БД (из Excel через load-data или из подготовленного app.db)

Опциональные

Компонент	Зачем
Ollama + модель llama3.2	LLM-подсказки марок (opt-in)
OpenCV (pip install -e ".[cv]")	препроцессинг сканов
EasyOCR / torch (".[ocr]")	fallback OCR; тяжело, не default
Git	обновление с GitHub (альтернатива — zip)

Ollama на рабочем ПК (факт)

Параметр	Значение
Каталог моделей	C:\Users\User\.ollama\models (= %USERPROFILE%\ollama\models)
URL	http://127.0.0.1:11434
Модель по умолчанию	llama3.2 (ollama pull llama3.2)
В приложении	GUI → 10. Настройки → каталог / модель / «Проверить Ollama»

Стандартный путь Ollama на Windows **не требует** переноса моделей на D:. Диск D (D:\ollama\models) — только если сознательно выносите веса с C:.

3. Функциональные возможности

3.1. Бизнес-цикл (ежедневно)

- 1 Заявка (.docx / PDF / текст)
- 2 → извлечение марок и организаций
- 3 → валидация (P0-P2) + правки оператора
- 4 → подтверждение (corrections)
- 5 → расчёт испытаний (прайс + правила)
- 6 → КП (Word) → заказ в БД
- 7 → заявка на испытания / пакет документов

Возможность	Описание
Извлечение	PDF, Word .docx, свободный текст
Марки кабелей	regex + семейства YAML + нормализация OCR-латиницы
Организации	заказчик / производитель / ИЛ и др.; human check обязателен
Расчёт	fixed , per_core , per_group , time_based ; климатические часы
КП / заявка / протокол	Word по шаблонам data/templates/
Пакет документов	папка: КП + заявка + протокол + summary
Справочники	испытания, марки, организации, test_mappings
Ассистент	детерминированный MarkCorrector + fuzzy; LLM — opt-in
Боевой опыт	export/import zip на dev

3.2. Word — основной канал заявок

Большинство заявок в **Word (.docx)** — это **предпочтительный** вход:

Как	python-docx : параграфы + таблицы → тот же пайплайн марок/организаций, что у PDF
Плюс	нет OCR , нет DPI, меньше «кракозябр»
Минус	только .docx (старый .doc — пересохранить); фигуры/текстбокс могут не извлекаться
Практика	марки обычно извлекаются лучше , чем со скана; организации — всё равно сверять

PDF-сканы: Tesseract, DPI **400**, опционально preprocess OpenCV.

3.3. Что «в коде», что «в БД»

Слой	Содержимое
Код	правила расчёта, парсеры, семейства (YAML), GUI, CLI
test_items	прайс: названия, цены, rule_type / params
test_mappings	фраза требования → код испытания
cable_marks / organizations	накопленные справочники (в бою — с нуля)
orders / calculations	история работы
corrections / snapshots	опыт для обучения пайплайна

4. Архитектура (для понимающего оператора/разработчика)

```
1  src/request_processor/
2      extraction/      # PDF, DOCX, OCR, организации, семейства
3      parsing/         # разбор марки (жилы, сечение, ТУ)
4      calculation/     # cost_calculator, test_rules, climatic
5      mapping/         # requirement_mapper
6      generation/      # КП, заявка Word
7      validation/      # human-in-the-loop, eval
8      assistant/       # MarkCorrector + Ollama
9      persistence/     # sqlite_repo, training_repo
10     training/        # battle_experience pack
11     ui/gui.py         # вкладки 1-10
```

Принцип: детерминированный парсер **первичен**; LLM только если включён и уверенность детерминированного слоя низкая; **запись в справочники — после человека.**

5. Политика данных при боевом запуске (согласовано)

Данные	Действие
Прайс (test_items)	оставить как есть
Правила расчёта	оставить (код + поля прайса)
test_mappings	оставить

Данные	Действие
cable_marks	очистить — собирать корректно с нуля
organizations	очистить — собирать корректно с нуля
orders / calculations / extractions	очищаются командой подготовки (ссылки на mark/org)

```

1 request-processor prepare-battle-db --yes
2 # backup: data\app.db.pre_battle_*.db
3 # затем zip: scripts\build_release_zip.ps1 -IncludeAppDb

```

Почему так: dev-марки/организации часто «грязные» (OCR, тестовые заявки). Чистые справочники в бою + правки оператора = качественный корпус для следующих версий. Прайс трогать нельзя — иначе ломается экономика расчёта.

6. Руководство по эксплуатации (оператор)

6.1. Запуск

- Ярлык: «**Обработка заявок на испытания кабелей**» (рабочий стол)
- или start_gui.bat
- создать ярлык: powershell -ExecutionPolicy Bypass -File scripts\create_desktop_shortcut.ps1

6.2. Типовой день

1. **Заявка** → Обзор → .docx / PDF → **Извлечь**
2. Проверить **марки** (💡 принять/отклонить)
3. Проверить **организации** (заказчик ≠ ИЛ; производитель с заявки)
4. **Подтвердить заявку** — пишутся corrections
5. **2. Расчёт** — испытания (вручную или «из заявки»)
6. **3. КП** — Word, заказ в БД
7. **4. Заказы** — заявка на испытания / **пакет документов**

6.3. Правила дисциплины

1. Не подтверждать вслепую.
2. Плохой скан → DPI 400, не «ещё раз надеяться».
3. Word предпочтительнее скана, если есть выбор.
4. LLM не отменяет проверку глазами.
5. Раз в неделю — **экспорт боевого опыта**.

6.4. Вкладки GUI (кратко)

#	Вкладка	Роль
1	Заявка	вход, извлечение, 💡, подтверждение
2	Расчёт	испытания, климатика, итог
3	КП	Word + заказ
4	Заказы	история, пакет
5–6	Марки / Организации	справочники (наполняются в бою)
7	Справочник	прайс-испытания
8	История	расчёты
10	Настройки	климатика, LLM, боевой опыт, путь пакетов, mappings

7. Боевой опыт: фиксация, формат, передача

7.1. Зачем отдельный пакет, а не вся `app.db`

Причина	Пояснение
Объём	БД + PDF растут; zip опыта компактный
Конфиденциальность	не тащим все PDF/заказы на чужой ПК «как получится»
Слияние	dev и work не должны затирать друг друга (<code>host_id</code> + префиксы)
Обучение	нужны правки и снимки , а не копия операционной истории

7.2. Что внутри zip

Элемент	Содержимое
<code>manifest.json</code>	<code>host_id</code> , дата, счётчики, комментарий оператора
<code>corrections/*.jsonl</code>	правки при «Подтвердить» + <code>assistant_*.jsonl</code>
<code>parse_snapshots/*.json</code>	снимки парсинга (A/B, проблемные кейсы)
<code>assistant_sessions.jsonl</code>	журнал ассистента (до лимита)
<code>test_mappings_used.json</code>	маппинги с <code>usage_count ≥ 1</code>

Не входит по умолчанию: полная `app.db`, исходные PDF, `rag_corpus`.

7.3. Как фиксируется в процессе работы

Действие оператора	Куда пишется
Подтвердить заявку	data/training/corrections/YYYYMMDD_*.jsonl
Принять/отклонить 💡	corrections/assistant_*.jsonl
Снимок парсинга	data/parse_snapshots/*.json
«Испытания из заявки»	test_mappings.usage_count++

7.4. Экспорт (рабочий ПК)

GUI: 10. Настройки → комментарий → **Экспорт опыта (zip)**...

CLI:

```
1 request-processor export-battle-experience --note "неделя 15.07, письма Word"
2 # полный архив, не только дельта:
3 request-processor export-battle-experience --full -o D:\backup\battle.zip
```

Перенос: флешка / сеть / почта (zip).

7.5. Импорт (ПК разработчика)

```
1 request-processor import-battle-experience D:\backup\battle_....zip
2 request-processor sync-corrections # если нужно явно
```

Дальше: анализ corrections, доработка regex/семейств, pytest, новый релиз.

7.6. Ритм

- **Раз в неделю** — обязательный экспорт.
- **После сложной/провальной заявки** — сразу (с комментарием в manifest).
- Дельта по умолчанию: только новое с прошлого экспорта.

8. LLM (Ollama) — роль в эксперименте

```
1 Марка OCR/текст
2   → MarkCorrector (правила + fuzzy по cable_marks)
3   → [если enabled и confidence низкая] Ollama llama3.2
4   → 💡 оператор Принять/Отклонить
5   → jsonl
```

Обязателен?	Нет для цикла КП
-------------	------------------

Модель	llama3.2
Риск	галлюцинации → только после human-in-the-loop
Путь весов (work)	C:\Users\User\.ollama\models

Оценка LLM — отдельный спринт после стабильного боя без ИИ (см. 40 R2).

9. Установка и обновление (ссылка)

Пошагово: 43 - Боевой запуск на рабочий ПК (v0.9.1) и INSTALL.md в репозитории.

Кратко:

- 1. zip (+ опционально подготовленный app.db с прайсом)
- 2. scripts\install.ps1
- 3. ярлык на стол
- 4. Ollama уже есть → указать каталог / ollama pull llama3.2
- 5. 1–2 заявки end-to-end

Обновление: сохранить data\app.db и training\corrections , новый zip / install.ps1 / migrate-db .

10. Критерии успеха эксперимента (1–4 недели)

#	Критерий	Как измерить
1	Цикл Word→КП→пакет без IDE	N заявок в неделю
2	Прайс стабилен, расчёты сходятся с ожиданием	сверка с ручным расчётом
3	Справочники марок/орг. растут корректно	выборочный аудит
4	Есть zip боевого опыта на dev	≥1 экспорт / неделя
5	Список багов/улучшений в Obsidian	заметки, не «в голове»
6	(опц.) LLM помог ≥2 «кривым» маркам	R2

Провал эксперимента: нельзя поставить на ПК; нет прайса; оператор не доверяет и уходит в Excel без export опыта.

11. Известные ограничения v0.9.1

- Нет автообучения «без человека».
 - Организации из текста/OCR — **ненадёжны** без проверки.
 - `.doc` (не `x`) не поддерживается.
 - Один `exe` (PyInstaller) — нет; `zip` + Python.
 - North Star (ТУ/ПМИ/Telegram/программа испытаний) — позже.
 - README на GitHub может отставать от паспорта — **истина: эта заметка + INSTALL.md**.
-

12. Чеклист «готов к экспериментальной эксплуатации»

- ☐ Python + `install.ps1` + GUI с ярлыка
 - ☐ `test_items` = актуальный прайс (не 3 демо)
 - ☐ `prepare-battle-db` выполнен **или** осознанно чистая БД + `load-data`
 - ☐ `cable_marks` / `organizations` пустые (старт с нуля)
 - ☐ Tesseract для PDF (если будут сканы)
 - ☐ Ollama: путь `C:\Users\User\.ollama\models`, модель `llama3.2` (если нужен LLM)
 - ☐ Оператор знает: Word → проверить → подтвердить → экспорт опыта
 - ☐ Канал передачи `zip` на dev согласован
-

13. Шпаргалка команд

```
1  # установка
2  powershell -ExecutionPolicy Bypass -File scripts\install.ps1
3  powershell -ExecutionPolicy Bypass -File
   scripts\create_desktop_shortcut.ps1
4
5  # БД к бою (прайс оставить)
6  request-processor prepare-battle-db --yes
7
8  # упаковка с БД
9  powershell -ExecutionPolicy Bypass -File scripts\build_release_zip.ps1 -
   IncludeAppDb
10
11 # LLM
12 ollama pull llama3.2
13 request-processor assistant-llm-status
14
15 # опыт
16 request-processor export-battle-experience --note "...
17 request-processor import-battle-experience path\to\battle.zip
```

Паспорт v0.9.1 для экспериментальной эксплуатации. Операционная «день 1» — 43.
Боевой опыт — 41.